

PHASE 4

Database Design and AI Integration

Chapters 5–6: data model, Gemini integration, prompts and AI safety

FITBUDDY • AI FITNESS PLAN GENERATOR • PROJECT REPORT

Chapter 5: Database Design

FitBuddy stores its data in SQLite, a self-contained, serverless relational database engine that keeps an entire database in a single file [4]. Access from Python is provided by SQLAlchemy, an object-relational mapper (ORM) that lets developers define tables as Python classes and query them with Python expressions instead of writing raw SQL for every operation [3].

5.1 Database Architecture

Persistence is organised around ORM models. Each model class corresponds to one table, and each attribute corresponds to a column. A database engine object is created from the configured DATABASE_URL; a session factory produces short-lived sessions that are opened for a request, used to read or write records, committed or rolled back, and closed. The database.py module centralises this configuration so that route handlers do not create engines or connections themselves.

Using an ORM has three benefits in this project: queries are expressed in the same language as the rest of the application; the mapping between objects and rows is defined in one place; and the underlying database can be changed with limited code modification because SQLAlchemy abstracts most dialect differences.

5.2 Entity Relationship Diagram

The data model contains three entities: User, WorkoutPlan and Feedback/Revision (implemented in this report as feedback_revisions). Figure 5.1 shows the entities, their attributes and their relationships.

Figure 5.1: Entity Relationship Diagram

Diagram to be inserted

One user can have many workout plans, and one workout plan can receive many feedback entries, each of which stores the corresponding revised plan. The feedback record also carries the user_id so that all feedback for a user can be retrieved directly. The original plan remains in the workout_plans table and is never modified by the revision workflow.

Tables 5.1 to 5.3 give the field-level description. The data types shown are typical SQLAlchemy choices and should be checked against the implemented models. [Verify types against database.py]

Table 5.1: users Table

Field	Type (typical)	Constraints	Description
id	Integer	Primary key, auto-increment	Internal row identifier
user_id	String	Required, unique, indexed	Application-level user identifier
name	String	Required	User name
age	Integer	Required	Age in years
weight	Float	Required	Body weight
goal	String	Required	Fitness goal
intensity	String	Required	Preferred workout intensity
created_at	DateTime	Default: current time	Record creation time
updated_at	DateTime	Updated on change	Last modification time

Table 5.2: workout_plans Table

Field	Type (typical)	Constraints	Description
id	Integer	Primary key	Plan identifier
user_id	String	Foreign key to users	Owner of the plan
original_plan	Text	Required	Original AI-generated plan (structured content)
nutrition_tip	Text	Optional	Nutrition/recovery tip from Gemini Flash
created_at	DateTime	Default: current time	Creation time
updated_at	DateTime	Updated on change	Last modification time

Table 5.3: feedback_revisions Table

Field	Type (typical)	Constraints	Description
id	Integer	Primary key	Feedback identifier
user_id	String	Foreign key to users	User who submitted the feedback
workout_plan_id	Integer	Foreign key to workout_plans	Plan to which the feedback refers
feedback	Text	Required	Free-text feedback from the user
revised_plan	Text	Required once generated	AI-generated revised plan
created_at	DateTime	Default: current time	Creation time

5.3 Data Integrity

- Primary keys uniquely identify each row in every table.
- Foreign keys connect plans to users and feedback to plans and users, so that related data can be retrieved consistently. SQLite enforces foreign keys only when the corresponding pragma is enabled for the connection; this should be confirmed in the implementation. [Verify]
- Relationships are declared in the ORM so that navigating from a user to plans and from a plan to feedback does not require manual joins.
- Required fields are declared as non-nullable so that incomplete records cannot be stored.
- Validation is performed by Pydantic before data reaches the database, providing a first line of defence; database constraints provide a second.
- Persistence is achieved by committing sessions after successful operations; failed operations are rolled back so that partial data is not saved.

5.4 Database Advantages

SQLite is appropriate for this project for several reasons. It requires no separate server process, which simplifies installation and demonstration. The database is a single file that is easy to back up, copy and inspect. It is included with Python, has mature tooling and provides full transactional guarantees adequate for a single-instance application with modest write concurrency.

Its limitations are equally clear: write concurrency is restricted, and it is not designed for many simultaneous application servers. For larger deployments, the recommended path is migration to a server database such as PostgreSQL. Because the application accesses data through SQLAlchemy, migration mainly involves changing the connection URL, installing the appropriate driver and introducing a migration tool such as Alembic to manage schema changes.

Chapter 6: AI Integration

6.1 Generative AI Overview

Generative AI refers to models that produce new content, such as text, in response to an input called a prompt. Modern large language models are based on the transformer architecture [8] and are trained on very large text collections so that, given a sequence of tokens, they can predict plausible continuations. With suitable instructions they can follow tasks described in natural language, including producing text in a requested structure, without task-specific retraining [9].

Two properties of such models are important for engineering. First, they are probabilistic: the same prompt can produce different outputs, so the application must be robust to variation. Second, they produce plausible text, not guaranteed-correct text; outputs must therefore be validated and, where the domain requires it, accompanied by appropriate cautions.

6.2 Gemini Integration

FitBuddy calls Google's Gemini models [5], [10] through the google-genai SDK [6], the current official Python SDK. The integration path is: FastAPI route -> application module -> google-genai client -> Gemini API -> generated response -> application processing. The client is created on the server with the API key read from an environment variable. Model identifiers are read from configuration, for example GEMINI_WORKOUT_MODEL and GEMINI_NUTRITION_MODEL, because available model names and versions change over time.

Figure 6.1: Workout Generation Flow

Diagram to be inserted

Figure 6.1 shows the workout-generation flow in detail. The validated profile is transformed into a prompt containing role, context, constraints and an output schema. The model response is checked for valid JSON with seven days; if it is not valid, the error-handling path returns a message rather than storing malformed data. The nutrition tip is then requested from the Gemini Flash model, and the plan and tip are stored together.

6.3 Workout Generation

The workout generator receives the user's age, weight, fitness goal and preferred intensity. These values are inserted into the prompt as context so that the model can adjust exercise selection, volume and rest to the profile. The expected output is a seven-day plan in which each day contains:

- a day number and a focus (for example upper body, lower body, cardio, mobility or rest);
- the planned duration in minutes;
- a list of exercises, each with sets, repetitions, rest period and brief notes;
- general notes such as warm-up, cool-down or safety reminders.

Requesting a full week, including rest or active-recovery days, matches how people actually schedule exercise and allows the frontend to render a uniform layout.

6.4 Nutrition Generation

The nutrition and recovery tip is produced by a separate call to a Gemini Flash model. Flash-class models are intended for low-latency, cost-efficient tasks, and a short tip does not require the depth of the workout-generation call. Separating the two calls also lets each prompt stay focused, allows different models to be configured independently, and means that a failure to produce a tip does not necessarily prevent the workout plan from being shown. The tip is intentionally general, covering hydration, balanced eating aligned with the goal, sleep and recovery, and does not constitute a clinical diet plan.

6.5 Feedback-Based Revision

When a user submits feedback, the system assembles three items: the original plan, the user feedback and the user profile, and sends them together to Gemini with an instruction to revise the plan accordingly while preserving the required structure. The model returns a revised plan, which is validated and stored as a new record linked to the original.

Figure 6.2: Feedback Revision Flow

Diagram to be inserted

Including the original plan lets the model make targeted changes instead of regenerating from scratch, which tends to keep unaffected days consistent. Including the profile prevents revisions from drifting outside the user's stated goal and intensity. Because only a new record is written, the original plan is untouched, and an administrator can compare the two versions.

6.6 Prompt Engineering

Prompts are designed with four elements:

- Role definition. The model is told to act as a fitness-planning assistant that produces general wellness guidance.
- Context. The user's age, weight, goal and intensity (and, for revisions, the original plan and feedback) are supplied.
- Constraints. The prompt requires exactly seven days, realistic durations, inclusion of rest or recovery, and avoidance of extreme or unsafe recommendations.
- Output structure. The prompt specifies JSON with named fields and forbids extra commentary outside the JSON.

A representative workout prompt is shown below. It illustrates the design and is not a verbatim copy of the implemented prompt. [Replace with actual prompt if required]

You are a fitness-planning assistant that provides general wellness guidance. Create a 7-day workout plan for the following user: Age: {age} Weight: {weight} Goal: {goal} Preferred intensity: {intensity} Constraints:

- Exactly 7 days, numbered 1 to 7; include rest or active-recovery days.
- Keep sessions realistic for the stated intensity.
- Do not give medical advice; avoid extreme or unsafe recommendations.

Return ONLY valid JSON matching the schema below, with no extra text. { "weekly_plan": [{ "day": 1, "focus": "...", "duration_minutes": 0, "exercises": [{ "name": "...", "sets": 0, "reps": "...", "rest_seconds": 0, "notes": "..." }] }, "notes": "..." }

A representative revision prompt follows the same pattern but adds the original plan and the feedback:

You are a fitness-planning assistant that provides general wellness guidance. User profile: age {age}, weight {weight}, goal {goal}, intensity {intensity}. Original plan (JSON): {original_plan} User feedback: {feedback} Revise the plan to address the feedback while respecting the profile. Keep exactly 7 days and the same JSON schema. Return ONLY valid JSON.

The API key is never placed in a prompt or in client-side code.

6.7 Structured AI Output

The application asks the model to respond in structured JSON. A simplified example of the expected structure is:

```
{ "weekly_plan": [ { "day": 1, "focus": "Upper Body", "duration_minutes": 40, "exercises": [] } ] }
```

Structured output improves the system in three ways. Frontend rendering becomes deterministic: the template can loop over days and exercises rather than displaying unpredictable free text. Data consistency improves because every plan has the same fields, which supports storage, comparison of original and revised versions and future analytics. Validation is possible: the backend can check that seven days are present and that required fields exist before saving. Because a model may still produce malformed output, the parsing step must handle errors, and it may strip stray formatting such as code fences before parsing. Some SDKs also allow a response schema or JSON response type to be requested, which further constrains output; the availability of such options depends on the SDK and model version in use.

6.8 AI Safety

AI-generated fitness advice raises specific safety concerns, and the system addresses them through design and disclosure:

- Output variability. The same input may yield different plans. Users are told that plans are suggestions, and revisions allow correction.
- Hallucinations. A model may state incorrect information confidently or invent unsuitable exercises. Prompts constrain the output and administrators can review stored plans.
- Fitness safety. Excessive volume, poor progression or unsuitable exercises can cause injury. Prompts instruct the model to avoid extreme recommendations, and plans should encourage warm-up, technique and rest.
- Medical limitations. The model has no access to medical history and the system does not screen for conditions. The application must display a disclaimer that it is not medical advice.
- Professional guidance. Users with health conditions, injuries, pregnancy or other concerns, and beginners uncertain about technique, should consult a qualified professional before starting a plan.